



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrian Davila Perez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 2

Integrante(s): 323001315, 426057857, 323008754, 426058665,
323067267

No. de lista o

brigada: 5

Semestre: 2027-1

Fecha de entrega: 4 de Septiembre

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Motivación	2
1.3. Objetivos	2
2. Marco Teórico	2
2.1. Variables y tipos de datos	2
2.2. Constantes	3
2.3. Expresiones y operadores	3
2.4. Estructuras de control	3
2.5. Instrucciones <code>break</code> y <code>continue</code>	3
3. Desarrollo	3
4. Resultados	7
5. Conclusiones	9

1. Introducción

1.1. Planteamiento del problema

Antes de abordar la Programación Orientada a Objetos es necesario dominar las bases de la programación estructurada: variables, condicionales y ciclos. Para reforzar estos conceptos se resolvieron tres ejercicios: el diseño de una aplicación para el control de aprobados y reprobados con un mecanismo de compensación, la determinación del mayor valor dentro de una serie de 10 enteros utilizando únicamente tres variables (contador, numero y mayor), y la generación de una tabla de valores mediante el uso de un ciclo.

1.2. Motivación

Resolver estos ejercicios permite fortalecer el pensamiento lógico-computacional y el manejo eficiente de estructuras de control, habilidades indispensables para comprender más adelante los principios de la POO, como la encapsulación y la reutilización de código.

1.3. Objetivos

- Aplicar estructuras de control (if, while y for) en la solución de problemas prácticos de programación.
- Implementar un sistema de conteo de aprobados y reprobados que incluya una lógica de compensación.
- Determinar el valor mas grande dentro de una serie de datos, optimizando el uso de variables.
- Generar tablas de valores mediante un ciclo, reforzando el manejo de la iteración y el formateo de salidas.

2. Marco Teórico

2.1. Variables y tipos de datos

Las variables son elementos utilizados para almacenar información que puede ser utilizada y modificada durante la ejecución de un programa. En Java, cada variable debe tener un tipo de dato que determine la clase de información que puede almacenar. Entre los tipos de datos básicos se encuentran `int` para números enteros, `double` para números decimales, `char` para caracteres y `boolean` para valores lógicos [1].

2.2. Constantes

Una constante es un dato cuyo valor permanece sin cambios durante la ejecución del programa. En Java, las constantes pueden declararse mediante la palabra reservada `final`, evitando que su valor sea modificado posteriormente. Su utilización permite mantener valores definidos que son necesarios para realizar determinadas operaciones [1].

2.3. Expresiones y operadores

Las expresiones permiten realizar operaciones utilizando variables, constantes y valores. Para construirlas se emplean operadores aritméticos, relacionales y lógicos. Los operadores aritméticos permiten realizar cálculos, mientras que los relacionales y lógicos permiten establecer condiciones que pueden ser evaluadas por el programa [1].

2.4. Estructuras de control

Las estructuras de control permiten determinar el orden en que se ejecutan las instrucciones de un programa. En Java se pueden utilizar estructuras condicionales como `if/else` y `switch`, las cuales permiten ejecutar diferentes instrucciones dependiendo de una condición o de un valor determinado [1].

También existen estructuras repetitivas como `while`, `do-while` y `for`, utilizadas para ejecutar un conjunto de instrucciones varias veces mientras se cumpla una determinada condición. Estas estructuras permiten controlar la repetición de procesos y evitar escribir las mismas instrucciones de manera repetitiva [1].

2.5. Instrucciones break y continue

Las instrucciones `break` y `continue` permiten modificar el comportamiento de los ciclos. `break` termina la ejecución del ciclo en el que se encuentra, mientras que `continue` hace que se omita la iteración actual y se continúe con la siguiente. Estas instrucciones permiten tener un mayor control sobre la ejecución de las estructuras repetitivas [1].

3. Desarrollo

Ejercicio 1

1. **Inicio del programa.** Se crea un objeto de la clase `Scanner` para permitir la lectura de datos desde el teclado.
2. **Declaración de variables.** Se declaran las variables `aprobados` y `reprobados` (inicializadas en 0), `contador` (inicializado en 1) y `resultado`.

3. **Control del ciclo.** Se establece un ciclo `while` que se repite mientras `contador` sea menor o igual a 10.
4. **Captura de datos.** Dentro del ciclo, se solicita al usuario ingresar un valor (1 = aprobó, 2 = reprobó), el cual se almacena en la variable `resultado`.
5. **Evaluación del resultado.**
 - a) Si `resultado` es igual a 1, se incrementa la variable `aprobados` en 1.
 - b) Si `resultado` es igual a 2, se incrementa la variable `reprobados` en 1.
6. **Incremento del contador.** Se incrementa `contador` en 1 y se regresa al Paso 3 para evaluar si el ciclo continúa.
7. **Fin del ciclo.** Una vez que `contador` supera el valor de 10, el ciclo termina.
8. **Impresión de resultados.** Se muestran en pantalla los totales acumulados de `aprobados` y `reprobados`.
9. **Evaluación de la bonificación.** Se verifica si `aprobados` es mayor o igual a 9.
[label=i.]
 - a) Si la condición se cumple, se muestra el mensaje "**¡Bono para el profesor!**".
 - b) Si no se cumple, el programa no muestra ningún mensaje adicional.
10. **Cierre del programa.** Se cierra el objeto `Scanner` y finaliza la ejecución.

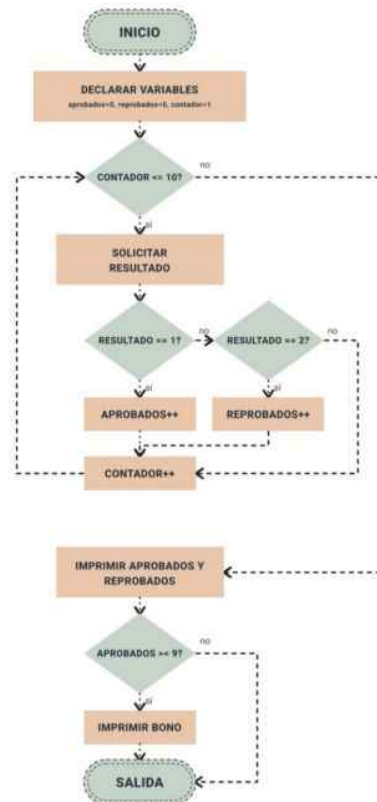


Figura 1: Diagrama de flujo Ejercicio 1

Ejercicio 2

1. **Inicio del programa.** Se crea un objeto de la clase `Scanner` para permitir la lectura de datos desde el teclado.
2. **Declaración de variables.** Se declaran las variables `contador`, `número` y `mayor`, inicializando `contador` en 1 y `mayor` en 0.
3. **Control del ciclo.** Se establece un ciclo `while` que se repite mientras `contador` sea menor o igual a 10.
4. **Captura de datos.** Dentro del ciclo, se solicita al usuario ingresar un número entero, el cual se almacena en la variable `número`.
5. **Cálculo del valor absoluto.** Se convierte `número` a su valor absoluto utilizando la función `Math.abs()`, para que los números negativos se traten como

positivos.

6. Comparación y actualización del mayor.

- a) Se verifica si **contador** es igual a 1, o si **número** es mayor que **mayor**.
- b) Si alguna de las dos condiciones se cumple, se asigna el valor de **número** a la variable **mayor**.
- c) Si ninguna condición se cumple, **mayor** conserva su valor actual.

7. Incremento del contador. Se incrementa **contador** en 1 y se regresa al Paso 3 para evaluar si el ciclo continúa.

8. Fin del ciclo. Una vez que **contador** supera el valor de 10, el ciclo termina.

9. Impresión del resultado. Se muestra en pantalla el valor final de **mayor**, correspondiente al número entero más grande de la serie ingresada.

10. Cierre del programa. Se cierra el objeto **Scanner** y finaliza la ejecución.

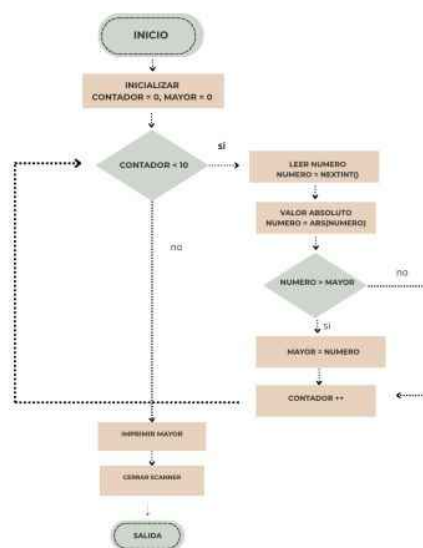


Figura 2: Diagrama de flujo del ejercicio 2

Ejercicio 3

1. **Inicio del programa.**
2. **Declaración de variables.** Se declara la variable n , que servirá como control del ciclo y base de los cálculos.
3. **Impresión del encabezado.** Se muestra en pantalla el título de cada columna: n , $10 \cdot n$, $100 \cdot n$ y $1000 \cdot n$.

4. **Control del ciclo.** Se establece un ciclo `for` que inicializa n en 1 y se repite mientras n sea menor o igual a 5, incrementando n en 1 al final de cada repetición.
5. **Cálculo de los valores.** En cada repetición, se calculan $n \cdot 10$, $n \cdot 100$ y $n \cdot 1000$.
6. **Impresión de la fila.** Se muestran en una misma línea los valores de n , $n \cdot 10$, $n \cdot 100$ y $n \cdot 1000$, formando así una fila de la tabla.
7. **Repetición del ciclo.** Se regresa al Paso 4 hasta que n supere el valor de 5.
8. **Fin del ciclo.** Una vez que n es mayor que 5, el ciclo termina.
9. **Fin del programa.**

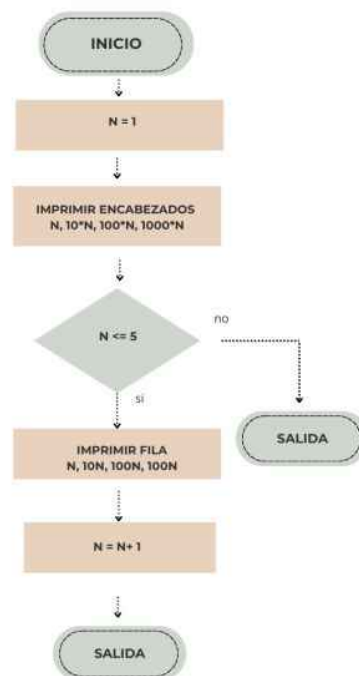


Figura 3: Diagrama de flujo del ejercicio 3

4. Resultados

1- El primer programa que se ejecuto fue donde se ingresaban 10 números según el usuario, buscaba entre esos 10 número y te daba el mayor.

```

PS C:\Users\acer\Desktop\P00> java Ejercicio2
Ingresa un numero: 10
Ingresa un numero: 9
Ingresa un numero: 4
Ingresa un numero: 8
Ingresa un numero: 200
Ingresa un numero: 390
Ingresa un numero: 1000
Ingresa un numero: 43
Ingresa un numero: 21
Ingresa un numero: 47
El numero mayor es: 1000
PS C:\Users\acer\Desktop\P00> |

```

Figura 4: Número mayor

2- El segundo programa imprime una tabla que muestra el resultado de multiplicar los números del 1 al 5 por 10, 100 y 1000. Utiliza un ciclo for para iterar esos cinco valores y calcular las operaciones matemáticas correspondientes en cada paso. Los datos se alinean visualmente en cuatro columnas bajo un encabezado estructurado mediante el uso de caracteres de tabulación (`\t`).

```

PS C:\Users\acer\Desktop\P00> java TablaValores
n      10*n    100*n   1000*n
1       10     100    1000
2       20     200    2000
3       30     300    3000
4       40     400    4000
5       50     500    5000
PS C:\Users\acer\Desktop\P00> |

```

Figura 5: Resultado de la Tabla de Valores

3- Aquí investigamos con nuestro LLM sobre el programa de reprobados y aprobados visto en clase y nos dio lo siguiente:

1. Ciclo de captura y clasificación: Un bucle `while` solicita 10 resultados y clasifica de inmediato si el alumno aprobó o reprobó utilizando contadores.

```

while (contador <= 10) {
    // Lectura del resultado desde consola
    if (resultado == 1) { aprobados++; }
    else if (resultado == 2) { reprobados++; }
    contador++;
}

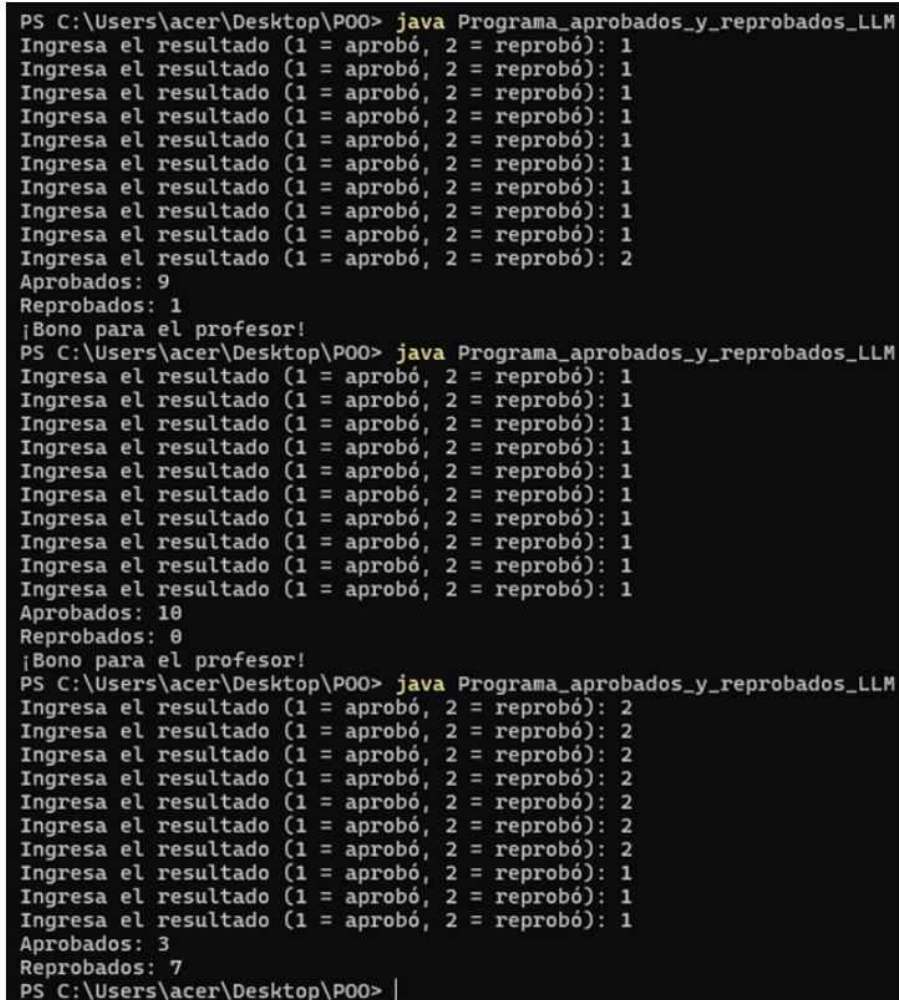
```

2. Condición de bonificación: Al terminar la captura, un condicional `if` evalúa si el total de alumnos aprobados cumple con el requisito mínimo para el bono.

```

if (aprobados >= 9) {
    System.out.println("¡Bono para el profesor!");
}

```



```

PS C:\Users\acer\Desktop\P00> java Programa_aprobados_y_reprobados_LLM
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Aprobados: 9
Reprobados: 1
¡Bono para el profesor!
PS C:\Users\acer\Desktop\P00> java Programa_aprobados_y_reprobados_LLM
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Aprobados: 10
Reprobados: 0
¡Bono para el profesor!
PS C:\Users\acer\Desktop\P00> java Programa_aprobados_y_reprobados_LLM
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 2
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Ingresa el resultado (1 = aprobó, 2 = reprobó): 1
Aprobados: 3
Reprobados: 7
PS C:\Users\acer\Desktop\P00> |

```

Figura 6: Ejecución del programa y resultados en consola.

5. Conclusiones

En esta práctica se cumplieron los objetivos planteados al utilizar estructuras de control como if, while y for para resolver los diferentes ejercicios. Se logró identificar el número mayor dentro de una serie de datos utilizando únicamente las variables necesarias, generar una tabla de valores mediante un ciclo y desarrollar un conteo de alumnos aprobados y reprobados incluyendo la condición para otorgar la bonificación.

Además, los ejercicios permitieron reforzar el uso de variables, operadores y ciclos para controlar el procesamiento y la repetición de datos. Con esto se comprendió mejor cómo organizar instrucciones sencillas en Java para obtener los resultados esperados,

reforzando las bases de programación estructurada necesarias para posteriormente trabajar con los conceptos de Programación Orientada a Objetos.

Referencias

Referencias

- [1] A. Martín, *Programador Certificado Java 2*, 2.a ed. México: Alfaomega Grupo Editor, 2008.